

Subject 2: Data Engineering for LLM Applications

1. The 80/20 Core (Hiring-Critical)

What Actually Matters (20% → 80% Impact)

Included (Production & Interview Critical):

1. **Document Chunking Strategy Engineering** (35% of value)
 - Semantic chunking vs. fixed-size vs. recursive splitting
 - Overlap optimization (information preservation vs. redundancy)
 - Chunk size impact on retrieval quality and cost
 - Why: Bad chunking = 40-60% retrieval accuracy loss, impossible to fix downstream
2. **Embedding Pipeline Design** (25% of value)
 - Batch processing vs. streaming
 - Embedding model selection (OpenAI vs. open-source, dimensionality tradeoffs)
 - Cost optimization (caching, deduplication)
 - Version management (model updates break similarity search)
 - Why: Embeddings are the foundation of RAG; wrong choices compound forever
3. **Vector Database Operations** (20% of value)
 - Index selection (HNSW, IVF, Flat - tradeoffs)
 - Metadata filtering (pre-filter vs. post-filter performance)
 - Hybrid search (vector + keyword)
 - Why: Database choice locks you in; wrong index = 10x latency
4. **Data Preprocessing for LLMs** (10% of value)
 - Text extraction from PDFs, DOCX, HTML (encoding hell)
 - Cleaning strategies (remove boilerplate, preserve structure)
 - Normalization (Unicode, whitespace, special characters)
 - Why: "Garbage in, garbage out" - LLMs amplify data quality issues
5. **Incremental Data Updates** (10% of value)
 - Change detection (delta processing vs. full reindex)
 - Embedding invalidation strategies
 - Versioning documents and embeddings together
 - Why: Full reindexing = expensive, slow; incremental = production-ready

Explicitly Excluded (Low ROI / Interview Anti-Signals):

- ❌ Traditional data warehousing (Redshift, Snowflake)
 - *Why excluded:* LLM apps rarely need OLAP; focus on retrieval, not analytics
- ❌ Apache Spark / distributed computing
 - *Why excluded:* Overkill for <10M documents; signals "big data" not "LLM engineering"
- ❌ Deep learning training pipelines
 - *Why excluded:* You're using pre-trained models, not training your own
- ❌ Feature engineering for ML models
 - *Why excluded:* LLMs use embeddings, not handcrafted features
- ❌ Real-time streaming (Kafka, Flink) as default
 - *Why excluded:* Batch processing sufficient for 90% of LLM apps; premature optimization

Hiring Signal Justification

What Tier 1 interviewers look for:

- Can you explain why chunk size matters and measure its impact?
- Do you understand vector database index tradeoffs (recall vs. latency)?
- Can you design an incremental update pipeline that doesn't reprocess everything?
- Have you debugged retrieval failures caused by bad chunking/embeddings?

What signals "student not engineer":

- "I used LangChain's default chunking" (no understanding of internals)
- "I just throw everything into Pinecone" (no index/filtering strategy)
- No mention of cost optimization (embeddings are expensive)
- Treating embeddings as magic (no versioning, caching, or monitoring)

2. Genius-Level Understanding

Core Mental Models

Perspective 1: Infrastructure Engineer

LLM data pipelines are inverted databases: optimize for retrieval, not storage.

Traditional database:

```
Write-optimized → Store normalized → Join on read
```

LLM data pipeline:

```
Read-optimized → Store denormalized → Pre-compute embeddings
```

Key insight: You pay the cost at write-time (chunking, embedding, indexing) to make retrieval fast and accurate.

Production implications:

python

```
# Amateur: Process on every query
def search(query):
    docs = load_all_documents() # Expensive
    chunks = chunk_documents(docs) # Expensive
    embeddings = embed_chunks(chunks) # VERY expensive
    return vector_search(query, embeddings)

# Engineer: Pre-process, optimize retrieval
class DataPipeline:
    def ingest(self, documents):
        """Run once, optimize heavily"""
        chunks = self.chunk_with_overlap(documents)
        embeddings = self.batch_embed(chunks) # Batch API
        self.vector_db.upsert(embeddings, metadata=chunks)

    def search(self, query):
        """Run millions of times, must be fast"""
        query_embedding = self.embed_query(query) # Cached
        return self.vector_db.search(query_embedding, top_k=10)
```

Cost example:

- Amateur approach: \$0.50 per search (re-embedding all docs)
- Engineer approach: \$0.0001 per search (pre-computed embeddings)
- **5000x cost difference**

Perspective 2: Product Engineer

Chunking is lossy compression. Your strategy determines what information survives.

The Chunking Trilemma: You can optimize for 2 of 3:

1. **Context preservation** (large chunks keep relationships)
2. **Retrieval precision** (small chunks reduce noise)
3. **Cost efficiency** (fewer chunks = less storage/compute)

Example: Legal contract analysis

Strategy 1: Large chunks (1000 tokens)

- ✓ Preserves clause relationships
- ✓ Lower storage cost (fewer chunks)
- ✗ Retrieves irrelevant sections (low precision)
- ✗ Exceeds context window when returning top-k results

Strategy 2: Small chunks (200 tokens)

- ✓ High precision retrieval
- ✓ Fits more results in context
- ✗ Loses cross-clause context
- ✗ High storage cost (5x more chunks)

Strategy 3: Hierarchical chunking (engineer solution)

python

```
class HierarchicalChunker:
    def chunk_document(self, doc):
        # Small chunks for retrieval
        small_chunks = self.split_by_semantic_boundary(doc, size=200)

        # Large chunks for context
        large_chunks = self.split_by_section(doc, size=1000)

        # Link them
        for small in small_chunks:
            small.parent_chunk_id = find_parent(large_chunks, small)

        return small_chunks, large_chunks
```

```
def retrieve(self, query):
    # Search small chunks (precision)
    matches = self.vector_db.search(query, chunk_type="small")

    # Return parent chunks (context)
    return [self.get_parent(m) for m in matches]
```

Result: Precision of small chunks + context of large chunks.

Perspective 3: Researcher

Embeddings are learned projections into semantic space. Chunking boundaries determine what gets projected.

Advanced insight: Chunk boundaries affect embedding quality

python

```
# Bad chunking: Breaks mid-sentence
chunk1 = "The company's revenue increased by 23% in Q4, driven by strong"
chunk2 = "sales in the APAC region. Customer acquisition costs decreased"

embed(chunk1) # Incomplete semantic unit - poor embedding
embed(chunk2) # Missing context - poor embedding

# Good chunking: Respects semantic boundaries
chunk1 = "The company's revenue increased by 23% in Q4, driven by strong sales in the APAC region"
chunk2 = "Customer acquisition costs decreased significantly compared to previous quarters"

embed(chunk1) # Complete thought - high-quality embedding
embed(chunk2) # Self-contained - high-quality embedding
```

Measurement:

python

```
from sentence_transformers import SentenceTransformer
import numpy as np

def measure_embedding_quality(chunk):
    """Higher entropy = more information captured"""
    embedding = model.encode(chunk)
    return np.std(embedding) # Higher variance = more signal

# Result:
```

```
# Bad chunks: std = 0.12 (low information)
# Good chunks: std = 0.34 (high information)
```

Why this matters: Retrieval quality is bounded by embedding quality, which is bounded by chunking quality.

Perspective 4: Hiring Manager

Show me your data pipeline architecture diagram, versioning strategy, and cost breakdown.

Red flags:

- "I used the default settings" (no engineering decisions)
- No incremental update strategy (full reindex on every change)
- No cost tracking (embeddings at \$0.0001/1k tokens × 10M chunks = \$1000)
- No data versioning (can't reproduce results)
- No monitoring (chunk size distribution, embedding cache hit rate)

Green flags:

- Architecture diagram showing ingestion → chunking → embedding → indexing flow
- Cost analysis: "Reduced embedding cost 70% via deduplication and caching"
- Incremental update logic: "Only reprocess changed documents"
- Data versioning: "Can rollback to any snapshot"
- Monitoring dashboard: chunk size distribution, embedding generation latency

Advanced Production Use Cases

Case 1: Semantic Chunking with Sentence Transformers

Problem: Fixed-size chunking breaks logical units mid-thought.

Solution: Use embedding similarity to detect semantic boundaries.

python

```
from sentence_transformers import SentenceTransformer
import numpy as np

class SemanticChunker:
    def __init__(self, threshold=0.7, max_chunk_size=500):
```

```

self.model = SentenceTransformer('all-MiniLM-L6-v2')
self.threshold = threshold
self.max_chunk_size = max_chunk_size

def chunk(self, text):
    sentences = self.split_sentences(text)
    embeddings = self.model.encode(sentences)

    chunks = []
    current_chunk = [sentences[0]]

    for i in range(1, len(sentences)):
        # Calculate similarity with previous sentence
        similarity = np.dot(embeddings[i-1], embeddings[i])

        if similarity < self.threshold or len(' '.join(current_chunk)) > self.max_ch
            # Start new chunk (semantic boundary detected)
            chunks.append(' '.join(current_chunk))
            current_chunk = [sentences[i]]
        else:
            current_chunk.append(sentences[i])

    chunks.append(' '.join(current_chunk))
    return chunks

```

Impact: 15-25% retrieval accuracy improvement vs. fixed-size chunking.

Cost: ~2x slower chunking (acceptable for batch processing).

Case 2: Embedding Deduplication for Cost Savings

Problem: Duplicate/near-duplicate chunks waste embedding API calls.

Solution: Hash-based deduplication + fuzzy matching.

python

```

import hashlib
from collections import defaultdict

class EmbeddingCache:
    def __init__(self, similarity_threshold=0.95):
        self.cache = {} # hash -> embedding
        self.text_to_hash = {} # exact matches
        self.fuzzy_index = [] # for near-duplicate detection
        self.threshold = similarity_threshold

```

```

def get_or_embed(self, text, embed_fn):
    # Exact match
    text_hash = hashlib.sha256(text.encode()).hexdigest()
    if text_hash in self.cache:
        return self.cache[text_hash], True # Cache hit

    # Check fuzzy matches
    embedding = embed_fn(text)

    for cached_text, cached_emb in self.fuzzy_index:
        similarity = np.dot(embedding, cached_emb)
        if similarity > self.threshold:
            # Near-duplicate found
            self.cache[text_hash] = cached_emb
            return cached_emb, True

    # New unique text
    self.cache[text_hash] = embedding
    self.fuzzy_index.append((text, embedding))
    return embedding, False

# Usage
cache = EmbeddingCache()
embeddings = []

for chunk in chunks:
    emb, was_cached = cache.get_or_embed(chunk, openai.embed)
    embeddings.append(emb)

print(f"Cache hit rate: {cache.hit_rate:.1%}")
# Typical result: 30-50% cache hit rate on real datasets

```

Impact: 30-50% reduction in embedding API costs.

Case 3: Hybrid Search for Improved Recall

Problem: Pure vector search misses exact keyword matches (names, codes, dates).

Solution: Combine vector similarity with BM25 keyword search.

python

```

from rank_bm25 import BM25Okapi
import numpy as np

```

```

class HybridSearcher:
    def __init__(self, vector_db, documents):
        self.vector_db = vector_db

        # Build BM25 index
        tokenized_docs = [doc.split() for doc in documents]
        self.bm25 = BM25Okapi(tokenized_docs)
        self.documents = documents

    def search(self, query, top_k=10, alpha=0.5):
        """
        alpha: weight for vector search (0-1)
        1-alpha: weight for keyword search
        """
        # Vector search
        vector_scores = self.vector_db.search(query, top_k=top_k*2)

        # Keyword search
        tokenized_query = query.split()
        bm25_scores = self.bm25.get_scores(tokenized_query)

        # Normalize scores
        vector_scores_norm = self.normalize(vector_scores)
        bm25_scores_norm = self.normalize(bm25_scores)

        # Combine
        combined_scores = (
            alpha * vector_scores_norm +
            (1 - alpha) * bm25_scores_norm
        )

        # Return top-k
        top_indices = np.argsort(combined_scores)[-top_k:][::-1]
        return [self.documents[i] for i in top_indices]

```

Impact: 10-20% recall improvement, especially for:

- Entity searches (product codes, names)
- Date-specific queries
- Rare terminology

When to use:

- Legal documents (case numbers, statute references)

- Technical documentation (API endpoints, error codes)
 - E-commerce (SKUs, brand names)
-

Expert-Level Comprehension Tests

Question 1: Your RAG system's retrieval accuracy dropped from 85% to 62% after reindexing with a new embedding model. Documents and queries are identical. What are your top 3 hypotheses and how do you diagnose each?

Expected Answer

Hypotheses: 1. **Embedding dimensionality mismatch with index** (most likely) - Diagnosis: Check vector DB index config vs. new model dimensions - Old model: 768-dim, New model: 1536-dim - Fix: Recreate index with correct dimensionality 2. **Normalized vs. unnormalized embeddings** - Diagnosis: Compute dot product vs. cosine similarity scores - Some models output normalized embeddings, others don't - Fix: Normalize embeddings before indexing, use cosine distance metric 3. **Changed semantic space** (different training data) - Diagnosis: Plot embedding PCA before/after, check cluster coherence - OpenAI ada-002 → ada-003 can have different semantic spaces - Fix: Re-tune similarity threshold, rebuild eval set with new model

Advanced diagnosis: python

```
# Compare embedding distributions
old_embs = load_old_embeddings()
new_embs = load_new_embeddings()

print(f"Old dims: {old_embs.shape[1]}, New dims: {new_embs.shape[1]}")
print(f"Old norm: {np.mean(np.linalg.norm(old_embs, axis=1))}")
print(f"New norm: {np.mean(np.linalg.norm(new_embs, axis=1))}")
```

Anti-pattern: "The new model is worse" (no systematic diagnosis)

Question 2: You have 1M documents to chunk and embed. The naive approach takes 48 hours and costs \$500. How do you optimize for <4 hours and <\$100?

Expected Answer

Optimization strategies: 1. **Batch API calls** (10x speedup, same cost) python

```
# Naive: 1 API call per chunk
for chunk in chunks:
    embedding = openai.embed(chunk) # 1M API calls

# Optimized: Batch requests
```

```
batch_size = 100
for i in range(0, len(chunks), batch_size):
    batch = chunks[i:i+batch_size]
    embeddings = openai.embed(batch) # 10k API calls
```

2. **Deduplication** (30-50% cost reduction) python

```
# Remove duplicate chunks before embedding
unique_chunks = set(chunks) # Exact matches
# Or use SimHash for near-duplicates (90%+ similarity)
```

3. **Caching** (20-40% cost reduction) python

```
# Check if chunks already embedded in previous runs
existing_embeddings = load_from_cache()
new_chunks = [c for c in chunks if c not in existing_embeddings]
```

4. **Parallel processing** (4-8x speedup) python

```
from concurrent.futures import ThreadPoolExecutor

with ThreadPoolExecutor(max_workers=10) as executor:
    embeddings = list(executor.map(embed_batch, chunk_batches))
```

5. **Use cheaper embedding model** (5-10x cost reduction) python

```
# GPT-3 ada: $0.0001/1k tokens
# Sentence-Transformers: Free (self-hosted)
# Tradeoff: Slight quality decrease (~5%)
```

Combined impact: - Batching: 48h → 5h - Parallelization: 5h → 1h - Deduplication: \$500 → \$300 - Cheaper model: \$300 → \$60 **Final: 1-2 hours, \$60-80**

Question 3: A user reports that searching for "Python debugging tools" returns results about snake handling. How would you debug and fix this?

Expected Answer

Diagnosis process: 1. **Inspect retrieved chunks** python

```
query = "Python debugging tools"
results = vector_db.search(query, top_k=10)
```

```

for r in results:
    print(f"Score: {r.score}, Text: {r.text[:100]}")

# Likely seeing:
# Score: 0.82, Text: "Snakes, including pythons, require careful handling..."

```

2. **Check embedding similarity** python

```

query_emb = embed("Python debugging tools")
snake_emb = embed("Python snake handling")
print(f"Similarity: {cosine_similarity(query_emb, snake_emb)}")
# Likely high due to word overlap "Python"

```

****Root cause:**** No domain context in embeddings. ****Fixes (in order of complexity):**** ****Fix 1:** Add metadata filtering** python

```

# Store document category during indexing
vector_db.upsert(
    embedding=emb,
    metadata={"category": "programming", "topic": "debugging"}
)

# Filter during search
results = vector_db.search(
    query,
    filter={"category": "programming"}
)

```

****Fix 2: Domain-specific embedding model**** python

```

# Use code-specific embeddings
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('microsoft/codebert-base')

# Or fine-tune on domain data

```

****Fix 3: Hybrid search with keyword filtering**** python

```

# Add BM25 component
results = hybrid_search(
    query,

```

```
required_keywords=["programming", "code", "software"]
)
```

****Fix 4: Query rewriting**** python

```
# Add domain context to query
rewritten_query = "Software development: Python debugging tools"
# Embedding now has stronger programming signal
```

****Long-term fix:**** Build domain-specific corpus, fine-tune embeddings.

3. Fastest Path to Competence (Execution-Focused)

Prerequisites (Week 0: Setup)

Install:

bash

```
pip install \
  sentence-transformers \ # Open-source embeddings
  chromadb \             # Vector database
  pypdf2 PyPDF2 \        # PDF parsing
  python-docx \          # DOCX parsing
  beautifulsoup4 \       # HTML parsing
  tiktoken \             # Token counting
  pandas numpy \         # Data manipulation
  rank-bm25 \            # Keyword search
  pytest                # Testing
```

Free resources:

- ChromaDB: Open-source vector DB (runs locally)
- Sentence-Transformers: Free embedding models
- Alternative: Qdrant (also open-source)

Repository structure:

```
llm-data-pipeline/
├── data/
│   ├── raw/           # Original documents
│   ├── processed/     # Chunked data
│   └── embeddings/    # Cached embeddings
├── src/
│   ├── chunking.py    # Chunking strategies
│   ├── embedding.py   # Embedding pipeline
│   ├── vector_db.py   # DB operations
│   └── preprocessing.py # Text cleaning
├── tests/
│   ├── test_chunking.py
│   └── test_retrieval.py
├── notebooks/
│   └── chunk_analysis.ipynb # Visual analysis
└── README.md
```

Week 1: Chunking Strategies & Quality Measurement

Learning Objectives:

- Implement 3+ chunking strategies
- Measure chunk quality with quantitative metrics
- Understand chunking impact on retrieval

Day 1-2: Fixed-Size Chunking Baseline

Hands-on Lab:

python

```
# src/chunking.py
import tiktoken

class FixedSizeChunker:
    def __init__(self, chunk_size=500, overlap=50):
        self.chunk_size = chunk_size
        self.overlap = overlap
        self.encoder = tiktoken.get_encoding("cl100k_base")

    def chunk(self, text):
        tokens = self.encoder.encode(text)
```

```

        chunks = []

        for i in range(0, len(tokens), self.chunk_size - self.overlap):
            chunk_tokens = tokens[i:i + self.chunk_size]
            chunk_text = self.encoder.decode(chunk_tokens)
            chunks.append({
                'text': chunk_text,
                'start_idx': i,
                'token_count': len(chunk_tokens)
            })

        return chunks

# Usage
chunker = FixedSizeChunker(chunk_size=300, overlap=50)
text = open('data/raw/document.txt').read()
chunks = chunker.chunk(text)

print(f"Total chunks: {len(chunks)}")
print(f"Avg tokens: {np.mean([c['token_count'] for c in chunks])}")

```

Deliverable: A notebook analyzing:

- Chunk size distribution
- Overlap impact on redundancy
- Broken sentences at boundaries

Day 3-4: Semantic Chunking

Hands-on Lab:

python

```

from sentence_transformers import SentenceTransformer
import numpy as np

class SemanticChunker:
    def __init__(self, threshold=0.75, max_chunk_size=500):
        self.model = SentenceTransformer('all-MiniLM-L6-v2')
        self.threshold = threshold
        self.max_chunk_size = max_chunk_size

    def split_sentences(self, text):
        """Simple sentence splitter"""
        import re

```

```

sentences = re.split(r'(?<=[.!?])\s+', text)
return sentences

def chunk(self, text):
    sentences = self.split_sentences(text)
    embeddings = self.model.encode(sentences)

    chunks = []
    current_chunk = []
    current_length = 0

    for i, sentence in enumerate(sentences):
        if i == 0:
            current_chunk.append(sentence)
            current_length += len(sentence.split())
            continue

        # Calculate similarity with previous sentence
        similarity = np.dot(embeddings[i-1], embeddings[i])

        # Check if we should start a new chunk
        if (similarity < self.threshold or
            current_length > self.max_chunk_size):
            chunks.append(' '.join(current_chunk))
            current_chunk = [sentence]
            current_length = len(sentence.split())
        else:
            current_chunk.append(sentence)
            current_length += len(sentence.split())

    if current_chunk:
        chunks.append(' '.join(current_chunk))

    return chunks

# Compare strategies
fixed_chunks = FixedSizeChunker().chunk(text)
semantic_chunks = SemanticChunker().chunk(text)

print(f"Fixed chunks: {len(fixed_chunks)}")
print(f"Sematic chunks: {len(semantic_chunks)}")

```

Deliverable: Comparison showing:

- Semantic coherence score (manual review of 20 random chunks)

- Chunk boundary quality (% ending mid-sentence)

Resources:

- [LangChain Text Splitters](#) - Free
- [Semantic Chunking Paper](#) - Theory

Day 5-7: Chunk Quality Metrics

Hands-on Lab:

Build metrics to evaluate chunking quality:

python

```
class ChunkQualityAnalyzer:
    def __init__(self, chunks, model):
        self.chunks = chunks
        self.model = model

    def measure_coherence(self):
        """Higher is better - measures semantic unity"""
        embeddings = self.model.encode([c['text'] for c in self.chunks])

        coherence_scores = []
        for i, chunk in enumerate(self.chunks):
            sentences = self.split_sentences(chunk['text'])
            if len(sentences) < 2:
                continue

            sent_embs = self.model.encode(sentences)
            # Average pairwise similarity within chunk
            similarities = []
            for j in range(len(sent_embs) - 1):
                sim = np.dot(sent_embs[j], sent_embs[j+1])
                similarities.append(sim)

            coherence_scores.append(np.mean(similarities))

        return np.mean(coherence_scores)

    def measure_boundary_quality(self):
        """% of chunks ending with proper punctuation"""
        proper_endings = sum(
            1 for c in self.chunks
            if c['text'].strip()[-1] in '!.?'
```

```

    )
    return proper_endings / len(self.chunks)

def measure_size_distribution(self):
    """Coefficient of variation (lower is more consistent)"""
    sizes = [len(c['text'].split()) for c in self.chunks]
    return np.std(sizes) / np.mean(sizes)

def generate_report(self):
    return {
        'coherence': self.measure_coherence(),
        'boundary_quality': self.measure_boundary_quality(),
        'size_cv': self.measure_size_distribution(),
        'total_chunks': len(self.chunks)
    }

# Compare strategies
model = SentenceTransformer('all-MiniLM-L6-v2')

fixed_quality = ChunkQualityAnalyzer(fixed_chunks, model).generate_report()
semantic_quality = ChunkQualityAnalyzer(semantic_chunks, model).generate_report()

print("Fixed-size chunking:", fixed_quality)
print("Semantic chunking:", semantic_quality)

```

Expected results:

```

Fixed-size:    coherence=0.68, boundary_quality=0.45, size_cv=0.12
Semantic:     coherence=0.82, boundary_quality=0.94, size_cv=0.31

```

Deliverable: A report documenting:

- Quantitative comparison of 3+ chunking strategies
- Recommendation with tradeoff analysis
- Visual plots of chunk size distributions

Week 2: Embedding Pipeline & Vector Database

Learning Objectives:

- Build a production embedding pipeline with caching
- Set up vector database with proper indexing

- Implement incremental updates

Day 8-10: Embedding Pipeline with Caching

Hands-on Lab:

python

```
# src/embedding.py
import hashlib
import pickle
from pathlib import Path
from sentence_transformers import SentenceTransformer

class EmbeddingPipeline:
    def __init__(self, model_name='all-MiniLM-L6-v2', cache_dir='data/embeddings'):
        self.model = SentenceTransformer(model_name)
        self.cache_dir = Path(cache_dir)
        self.cache_dir.mkdir(exist_ok=True)

        # Load cache index
        self.cache_index = self._load_cache_index()

    def _load_cache_index(self):
        index_path = self.cache_dir / 'index.pkl'
        if index_path.exists():
            with open(index_path, 'rb') as f:
                return pickle.load(f)
        return {}

    def _save_cache_index(self):
        with open(self.cache_dir / 'index.pkl', 'wb') as f:
            pickle.dump(self.cache_index, f)

    def _get_cache_key(self, text):
        return hashlib.sha256(text.encode()).hexdigest()

    def embed(self, texts, use_cache=True):
        """Embed texts with caching"""
        if isinstance(texts, str):
            texts = [texts]

        embeddings = []
        texts_to_embed = []
        text_indices = []
```

```

for i, text in enumerate(texts):
    cache_key = self._get_cache_key(text)

    if use_cache and cache_key in self.cache_index:
        # Load from cache
        cache_file = self.cache_dir / f"{cache_key}.npy"
        embedding = np.load(cache_file)
        embeddings.append((i, embedding))
    else:
        texts_to_embed.append(text)
        text_indices.append(i)

# Batch embed new texts
if texts_to_embed:
    new_embeddings = self.model.encode(texts_to_embed, batch_size=32)

# Cache new embeddings
for text, embedding, idx in zip(texts_to_embed, new_embeddings, text_indices):
    cache_key = self._get_cache_key(text)
    cache_file = self.cache_dir / f"{cache_key}.npy"

    np.save(cache_file, embedding)
    self.cache_index[cache_key] = str(cache_file)
    embeddings.append((idx, embedding))

# Save updated index
self._save_cache_index()

# Sort by original order
embeddings.sort(key=lambda x: x[0])
return np.array([emb for _, emb in embeddings])

def get_stats(self):
    return {
        'cached_embeddings': len(self.cache_index),
        'cache_size_mb': sum(
            f.stat().st_size for f in self.cache_dir.glob('*.npy')
        ) / 1024 / 1024
    }

# Usage
pipeline = EmbeddingPipeline()

# First run - embeds everything
chunks_text = [c['text'] for c in chunks]

```

```

embeddings = pipeline.embed(chunks_text)
print(f"Embedded {len(embeddings)} chunks")

# Second run - uses cache
embeddings_cached = pipeline.embed(chunks_text)
print(pipeline.get_stats())
# Output: {'cached_embeddings': 142, 'cache_size_mb': 2.3}

```

Deliverable: Demonstrate:

- 100x speedup on second run (cache hit)
- Cost tracking (if using paid API)

Day 11-12: Vector Database Setup

Hands-on Lab:

python

```

# src/vector_db.py
import chromadb
from chromadb.config import Settings

class VectorDatabase:
    def __init__(self, persist_directory='data/vector_db'):
        self.client = chromadb.Client(Settings(
            persist_directory=persist_directory,
            anonymized_telemetry=False
        ))

        # Create or get collection
        self.collection = self.client.get_or_create_collection(
            name="documents",
            metadata={"hnsw:space": "cosine"} # Use cosine similarity
        )

    def add_documents(self, chunks, embeddings, metadata=None):
        """Add documents with embeddings"""
        ids = [str(i) for i in range(len(chunks))]

        self.collection.add(
            ids=ids,
            embeddings=embeddings.tolist(),
            documents=[c['text'] for c in chunks],
            metadatas=metadata or [{}] * len(chunks)
        )

```

```

    )

    def search(self, query_embedding, top_k=10, filter=None):
        """Search for similar documents"""
        results = self.collection.query(
            query_embeddings=[query_embedding.tolist()],
            n_results=top_k,
            where=filter # Metadata filtering
        )

        return [
            {
                'id': results['ids'][0][i],
                'text': results['documents'][0][i],
                'distance': results['distances'][0][i],
                'metadata': results['metadatas'][0][i]
            }
            for i in range(len(results['ids'][0]))
        ]

    def delete_by_metadata(self, filter):
        """Delete documents matching filter"""
        self.collection.delete(where=filter)

    def get_stats(self):
        return {
            'total_documents': self.collection.count(),
            'collection_name': self.collection.name
        }

# Usage
db = VectorDatabase()

# Add documents with metadata
metadata = [
    {'source': 'doc1.pdf', 'page': i // 10, 'category': 'technical'}
    for i in range(len(chunks))
]

db.add_documents(chunks, embeddings, metadata)

# Search with filtering
query_emb = pipeline.embed("How do I debug Python code?")
results = db.search(
    query_emb,
    top_k=5,

```

```

        filter={'category': 'technical'}
    )

    for r in results:
        print(f"Score: {1-r['distance']:.3f}, Text: {r['text'][:100]}")

```

Deliverable: A working RAG system:

python

```

def rag_search(query):
    # Embed query
    query_emb = pipeline.embed(query)

    # Retrieve relevant chunks
    results = db.search(query_emb, top_k=3)

    # Format context
    context = '\n\n'.join([r['text'] for r in results])

    # Generate response
    prompt = f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"
    # ... call LLM with prompt

    return response

```

Resources:

- [ChromaDB Docs](#) - Free, local vector DB
- [Qdrant Tutorial](#) - Alternative vector DB

Day 13-14: Incremental Updates

Hands-on Lab:

python

```

class IncrementalPipeline:
    def __init__(self, vector_db, embedding_pipeline):
        self.db = vector_db
        self.embedder = embedding_pipeline
        self.document_index = self._load_document_index()

    def _load_document_index(self):

```

```

        """Track which documents have been processed"""
        # Simple JSON file: {doc_id: {hash, timestamp, chunk_ids}}
        index_path = Path('data/document_index.json')
        if index_path.exists():
            with open(index_path) as f:
                return json.load(f)
        return {}

    def _save_document_index(self):
        with open('data/document_index.json', 'w') as f:
            json.dump(self.document_index, f)

    def _hash_document(self, doc_path):
        """Hash file content"""
        with open(doc_path, 'rb') as f:
            return hashlib.sha256(f.read()).hexdigest()

    def process_document(self, doc_path):
        """Process new or updated document"""
        doc_id = str(doc_path)
        current_hash = self._hash_document(doc_path)

        # Check if already processed
        if doc_id in self.document_index:
            if self.document_index[doc_id]['hash'] == current_hash:
                print(f"Skipping {doc_id} (unchanged)")
                return
            else:
                print(f"Updating {doc_id} (changed)")
                # Delete old chunks
                old_chunk_ids = self.document_index[doc_id]['chunk_ids']
                for chunk_id in old_chunk_ids:
                    self.db.collection.delete(ids=[chunk_id])
        else:
            print(f"Adding new document {doc_id}")

        # Process document
        text = open(doc_path).read()
        chunks = chunker.chunk(text)
        embeddings = self.embedder.embed([c['text'] for c in chunks])

        # Generate unique chunk IDs
        chunk_ids = [f"{doc_id}_chunk_{i}" for i in range(len(chunks))]

        # Add to vector DB
        metadata = [{'source': doc_id, 'chunk_idx': i} for i in range(len(chunks))]

```

```

        self.db.collection.add(
            ids=chunk_ids,
            embeddings=embeddings.tolist(),
            documents=[c['text'] for c in chunks],
            metadatas=metadata
        )

    # Update index
    self.document_index[doc_id] = {
        'hash': current_hash,
        'timestamp': time.time(),
        'chunk_ids': chunk_ids
    }
    self._save_document_index()

    def process_directory(self, dir_path):
        """Process all documents in directory"""
        for doc_path in Path(dir_path).glob('*.txt'):
            self.process_document(doc_path)

# Usage
incremental = IncrementalPipeline(db, pipeline)

# First run - processes all
incremental.process_directory('data/raw')

# Modify one file
# ... edit data/raw/doc1.txt ...

# Second run - only reprocesses changed file
incremental.process_directory('data/raw')
# Output:
# Skipping data/raw/doc2.txt (unchanged)
# Skipping data/raw/doc3.txt (unchanged)
# Updating data/raw/doc1.txt (changed)

```

Deliverable: Demonstrate:

- 10x faster reindexing (only changed docs)
 - Document versioning (can list all versions)
-

Week 3: Production Data Processing & Portfolio

Learning Objectives:

- Handle real-world document formats (PDF, DOCX, HTML)
- Build robust preprocessing pipelines
- Create production-ready portfolio project

Day 15-17: Multi-Format Document Processing

Hands-on Lab:

python

```
# src/preprocessing.py
from pypdf import PdfReader
from docx import Document
from bs4 import BeautifulSoup
import re

class DocumentProcessor:
    def extract_text(self, file_path):
        """Extract text from various formats"""
        suffix = Path(file_path).suffix.lower()

        if suffix == '.pdf':
            return self._extract_pdf(file_path)
        elif suffix == '.docx':
            return self._extract_docx(file_path)
        elif suffix == '.html':
            return self._extract_html(file_path)
        elif suffix == '.txt':
            return self._extract_txt(file_path)
        else:
            raise ValueError(f"Unsupported format: {suffix}")

    def _extract_pdf(self, path):
        reader = PdfReader(path)
        text = []

        for page in reader.pages:
            text.append(page.extract_text())

        return '\n\n'.join(text)
```

```

def _extract_docx(self, path):
    doc = Document(path)
    return '\n\n'.join([p.text for p in doc.paragraphs])

def _extract_html(self, path):
    with open(path) as f:
        soup = BeautifulSoup(f.read(), 'html.parser')

        # Remove script and style elements
        for script in soup(["script", "style"]):
            script.decompose()

    return soup.get_text(separator='\n')

def _extract_txt(self, path):
    with open(path, 'r', encoding='utf-8') as f:
        return f.read()

def clean_text(self, text):
    """Clean extracted text"""
    # Remove multiple newlines
    text = re.sub(r'\n{3,}', '\n\n', text)

    # Remove multiple spaces
    text = re.sub(r' {2,}', ' ', text)

    # Fix common PDF extraction issues
    text = text.replace('fi', 'fi') # Ligatures
    text = text.replace('fl', 'fl')

    # Normalize unicode
    text = text.encode('utf-8', errors='ignore').decode('utf-8')

    return text.strip()

def process_file(self, path):
    """Full processing pipeline"""
    raw_text = self.extract_text(path)
    clean_text = self.clean_text(raw_text)

    return {
        'source': str(path),
        'text': clean_text,
        'char_count': len(clean_text),
        'word_count': len(clean_text.split())
    }

```

```
# Usage
processor = DocumentProcessor()

# Process different file types
for file_path in Path('data/raw').glob('*'):
    doc = processor.process_file(file_path)
    print(f"{doc['source']}: {doc['word_count']} words")
```

Deliverable: Process 10+ diverse documents (PDF, DOCX, HTML) and demonstrate:

- Successful text extraction
- Proper cleaning (before/after examples)
- Error handling for malformed files

Resources:

- [PyPDF2 vs pypdf](#) - PDF parsing
- [python-docx](#) - DOCX parsing

Day 18-19: End-to-End Pipeline

Hands-on Lab:

Build complete data pipeline:

python

```
# pipeline.py
class ProductionPipeline:
    def __init__(self):
        self.processor = DocumentProcessor()
        self.chunker = SemanticChunker()
        self.embedder = EmbeddingPipeline()
        self.db = VectorDatabase()
        self.incremental = IncrementalPipeline(self.db, self.embedder)

    def ingest(self, file_path):
        """Full ingestion pipeline"""
        print(f"Processing {file_path}")

        # 1. Extract and clean
        doc = self.processor.process_file(file_path)

        # 2. Chunk
```

```

        chunks = self.chunker.chunk(doc['text'])
        print(f"    Created {len(chunks)} chunks")

    # 3. Embed (with caching)
    embeddings = self.embedder.embed([c for c in chunks])

    # 4. Store in vector DB
    metadata = [
        {
            'source': doc['source'],
            'chunk_idx': i,
            'word_count': len(c.split())
        }
        for i, c in enumerate(chunks)
    ]

    self.db.add_documents(
        [{'text': c} for c in chunks],
        embeddings,
        metadata
    )

    print(f"    Indexed {len(chunks)} chunks")
    return len(chunks)

def search(self, query, top_k=5):
    """Search pipeline"""
    query_emb = self.embedder.embed(query)
    results = self.db.search(query_emb, top_k=top_k)

    return [
        {
            'text': r['text'],
            'source': r['metadata']['source'],
            'score': 1 - r['distance']
        }
        for r in results
    ]

def get_statistics(self):
    """Pipeline statistics"""
    return {
        'total_documents': self.db.get_stats()['total_documents'],
        'cached_embeddings': self.embedder.get_stats()['cached_embeddings'],
        'cache_size_mb': self.embedder.get_stats()['cache_size_mb']
    }

```

```
# Usage
pipeline = ProductionPipeline()

# Ingest directory
for file in Path('data/raw').glob('*'):
    pipeline.ingest(file)

# Search
results = pipeline.search("How do I configure logging?")
for r in results:
    print(f"[{r['score']:.3f}] {r['source']}: {r['text'][:100]}")

# Stats
print(pipeline.get_statistics())
```

Deliverable: A production-ready pipeline that:

- Handles multiple file formats
- Supports incremental updates
- Has caching for efficiency
- Provides usage statistics

Day 20-21: Portfolio Documentation

Create comprehensive README:

markdown

```
# Production LLM Data Pipeline

## Overview
End-to-end data pipeline for RAG systems, optimized for cost and quality.

## Architecture
```

Documents → Extraction → Cleaning → Chunking → Embedding → Vector DB

↓↓↓↓↓

(PDF/DOCX) (Unicode) (Semantic) (Cached) (ChromaDB)

```
## Features
- ✅ Multi-format support (PDF, DOCX, HTML, TXT)
- ✅ Semantic chunking (15% better retrieval vs fixed-size)
```

-  Embedding caching (100x faster on re-runs)
-  Incremental updates (10x faster than full reindex)
-  Metadata filtering for hybrid search
-  Production monitoring (chunk size, cache hit rate)

Performance Metrics

Metric	Baseline	Optimized	Improvement
Chunking quality (coherence)	0.68	0.82	+21%
Embedding cost (1M chunks)	\$500	\$80	84% reduction
Reindex time (100 docs)	48h	1.2h	40x faster
Cache hit rate	0%	68%	-

Retrieval Quality

Tested on 100 queries across technical documentation:

Strategy	MRR@5	Recall@10
Fixed-size chunking	0.62	0.71
Semantic chunking	0.78	0.85
+ Hybrid search	0.84	0.91

Usage

```
```python
from pipeline import ProductionPipeline

pipeline = ProductionPipeline()
pipeline.ingest('data/docs/manual.pdf')

results = pipeline.search("How do I configure logging?")
```

## Cost Analysis

---

Per 1M documents (avg 10 pages each):

- Document processing: Free (local)
- Chunking: Free (local)
- Embeddings: \$80 (all-MiniLM-L6-v2, local) or \$500 (OpenAI ada-002)
- Storage: \$0.12/month (ChromaDB local) or \$25/month (Pinecone)

**Total: \$80-525 one-time + \$0.12-25/month**

# Design Decisions

---

## Why Semantic Chunking?

Fixed-size chunking breaks mid-thought, reducing retrieval precision by 20-30%.  
Semantic chunking respects natural boundaries, improving coherence.

**Tradeoff:** 2x slower chunking (acceptable in batch processing).

## Why Local Embeddings?

all-MiniLM-L6-v2 provides 95% of OpenAI ada-002 quality at zero marginal cost.

**Tradeoff:** 5% lower retrieval quality (measured on our eval set).

## Why ChromaDB?

Simple, local-first, open-source. Sufficient for <10M documents.

**When to switch to Pinecone/Weaviate:**

- *10M documents*
- Multi-region deployment
- Sub-50ms latency requirements

## Future Improvements

---

1. Fine-tune embedding model on domain data (+10% retrieval quality)
2. Implement recursive chunking for structured documents
3. Add query rewriting for better retrieval

...

...

---

## 4. Blind Spots & Underrated Perspectives

---

### What's Poorly Taught (Industry Reality Gaps)

#### Blind Spot 1: Chunking is Domain-Dependent (No Universal Strategy)

**Common teaching:** "Use 500 tokens with 50 token overlap."

**Reality:** Optimal chunking depends entirely on document structure and query patterns.

**Examples:**

**Legal documents:**

python

```
Bad: Fixed-size chunks break legal clauses
"Section 3.2: The party shall... [CHUNK BREAK] ...indemnify all damages"
Retrieved chunk is legally incomplete

Good: Clause-aware chunking
def chunk_legal(doc):
 sections = doc.split('Section ')
 return [s for s in sections if validate_legal_completeness(s)]
```

**Code documentation:**

python

```
Bad: Splitting code examples mid-function
Chunk 1: "def process(data):\n # Clean data"
Chunk 2: " return cleaned\n"

Good: Keep functions intact
def chunk_code_docs(doc):
 chunks = []
 current = []
 in_code_block = False

 for line in doc.split('\n'):
 if line.startswith('```'):
 in_code_block = not in_code_block

 current.append(line)
```

```

 if not in_code_block and len('\n'.join(current)) > 500:
 chunks.append('\n'.join(current))
 current = []

 return chunks

```

### Conversational transcripts:

python

```

Bad: Breaking mid-conversation
Chunk 1: "User: How do I reset my password?\nAgent: Sure! First,"
Chunk 2: "click on Settings..."

Good: Keep Q&A pairs together
def chunk_conversations(transcript):
 turns = transcript.split('\n\n')
 chunks = []
 current_exchange = []

 for turn in turns:
 if turn.startswith('User:'):
 if current_exchange:
 chunks.append('\n'.join(current_exchange))
 current_exchange = [turn]
 else:
 current_exchange.append(turn)

 return chunks

```

### Why this matters for hiring:

- Shows domain expertise beyond "using LangChain defaults"
- Demonstrates problem-solving (adapting to data structure)
- Signals production experience (has encountered these issues)

### Blind Spot 2: Embedding Models Are Not One-Size-Fits-All

**Common teaching:** "Use OpenAI text-embedding-ada-002."

**Reality:** Embedding model choice has massive quality/cost tradeoffs.

**Decision matrix:**

Model	Dimensions	Cost	Best For
OpenAI ada-002	1536	\$0.0001/1k	General purpose, budget available
OpenAI ada-003	3072	\$0.00013/1k	Higher quality needed
all-MiniLM-L6-v2	384	Free	Cost-constrained, high-volume
gte-large	1024	Free	Better quality than MiniLM
Cohere embed-v3	1024	\$0.0001/1k	Multilingual
voyage-02	1024	\$0.0001/1k	Code/technical docs

### Hidden costs most engineers miss:

python

```
Cost calculation
docs = 1_000_000 # 1M documents
avg_tokens_per_doc = 500

OpenAI ada-002
cost_openai = (docs * avg_tokens_per_doc / 1000) * 0.0001
print(f"OpenAI: ${cost_openai}") # $50

Local model (all-MiniLM)
cost_local = 0 # One-time GPU cost or free on CPU
print(f"Local: ${cost_local}") # $0

BUT: Storage costs differ
ada-002: 1536 dims × 4 bytes × 1M = 6.1 GB
MiniLM: 384 dims × 4 bytes × 1M = 1.5 GB

Vector DB storage costs (Pinecone)
storage_cost_ada = 6.1 * 0.25 # $0.25/GB/month
storage_cost_mini = 1.5 * 0.25

print(f"Storage cost (ada): ${storage_cost_ada:.2f}/month") # $1.53/month
print(f"Storage cost (mini): ${storage_cost_mini:.2f}/month") # $0.38/month
```

### Quality tradeoffs (measured on MTEB benchmark):

Retrieval accuracy:

- OpenAI ada-002: 0.496
- gte-large: 0.522 (better, free!)
- all-MiniLM-L6-v2: 0.448 (slightly worse, free)

**When strong engineers choose what:**

- **Startups (<1M docs):** all-MiniLM-L6-v2 (quality good enough, cost = \$0)
- **Enterprise (>10M docs):** OpenAI ada-002 (quality matters more than cost)
- **Code search:** voyage-02 or Cohere (specialized)
- **Multilingual:** Cohere embed-v3 (best multilingual support)

### Blind Spot 3: Vector Databases Are Glorified Approximate Nearest Neighbors (With Tradeoffs)

**Common teaching:** "Use Pinecone, it's the best."

**Reality:** Vector DB choice depends on scale, latency, and cost requirements.

**The accuracy/speed/cost trilemma:**

python

```
Index types and their tradeoffs

1. Flat (Exact search)
✓ 100% recall
✗ O(n) search time - slow for >100k vectors
Use: <100k vectors, quality critical

2. HNSW (Hierarchical Navigable Small World)
✓ 95-99% recall with proper tuning
✓ Fast: O(log n)
✗ High memory usage (graph structure)
Use: <10M vectors, fast retrieval needed

3. IVF (Inverted File Index)
✓ Memory efficient
✓ Tunable recall/speed (nprobe parameter)
✗ 85-95% recall (misses some results)
Use: >10M vectors, cost-constrained
```

**Concrete example:**

python

```
import chromadb
from chromadb.config import Settings

HNSW index (default in ChromaDB)
collection_hnsw = client.create_collection(
 name="docs_hnsw",
 metadata={
 "hnsw:space": "cosine",
 "hnsw:construction_ef": 200, # Build quality
 "hnsw:search_ef": 100, # Search quality
 "hnsw:M": 16 # Connections per node
 }
)

Tuning parameters:
- High M (32-64): Better recall, more memory
- Low M (8-16): Faster, less memory
- High search_ef (100-500): Better recall, slower
- Low search_ef (10-50): Faster, lower recall

Test recall
true_neighbors = exact_search(query, top_k=10)
approx_neighbors = collection_hnsw.query(query, n=10)

recall = len(set(true_neighbors) & set(approx_neighbors)) / 10
print(f"Recall@10: {recall:.2%}") # Typical: 95-98%
```

### Production debugging scenario:

Problem: Vector search returns irrelevant results

Diagnosis checklist:

1.  Check embedding model (same for index and query?)
2.  Check normalization (cosine requires normalized vectors)
3.  Check distance metric (cosine vs. L2 vs. dot product)
4.  Check index recall (might be approximate search issue)

Solution: Increase HNSW search\_ef from 50 to 200

Result: Recall improved from 88% to 97%

Tradeoff: Latency increased from 15ms to 45ms (acceptable)

## Blind Spot 4: Metadata Filtering is a Performance Minefield

**Common teaching:** "Add metadata to chunks for filtering."

**Reality:** Pre-filter vs. post-filter has 10-100x performance implications.

**The problem:**

python

```
Naive approach (post-filter)
results = vector_db.search(query, top_k=10)
filtered = [r for r in results if r.metadata['category'] == 'technical']
❌ Problem: Might return 0-3 results after filtering (asked for 10!)

Engineer approach (pre-filter)
results = vector_db.search(
 query,
 top_k=10,
 filter={'category': 'technical'} # DB-level filtering
)
✅ Always returns 10 technical documents
```

**Performance implications:**

python

```
Benchmark: 1M documents, filter selects 10%

Post-filter approach
results = db.search(query, top_k=100) # Overquery to compensate
filtered = [r for r in results if matches_filter(r)][:10]
Latency: ~150ms (retrieves 100, filters in Python)

Pre-filter approach (metadata index)
results = db.search(query, top_k=10, filter=metadata_filter)
Latency: ~20ms (DB filters natively)

7.5x speedup
```

**Not all vector DBs support pre-filtering efficiently:**

python

```
ChromaDB: Native metadata filtering ✅
collection.query(
 query_embeddings=[emb],
 where={"category": "technical"} # Efficient
)

FAISS: No native filtering ❌
Must implement manually (slow)
results = index.search(query, k=1000)
filtered = [r for r in results if metadata[r.id]['category'] == 'technical'][:10]
```

### Strong engineer solution: Separate indices per category

python

```
Instead of one large index with metadata
large_index = {"all documents": 1_000_000}

Create category-specific indices
indices = {
 "technical": VectorDB("technical_docs"), # 100k docs
 "marketing": VectorDB("marketing_docs"), # 200k docs
 "legal": VectorDB("legal_docs") # 50k docs
}

Search only relevant index
results = indices['technical'].search(query, top_k=10)
✅ Faster (smaller index)
✅ No filtering overhead
❌ More infrastructure (3 DBs instead of 1)
```

### Blind Spot 5: Data Versioning is Critical (and Nobody Does It)

**Common teaching:** Not mentioned at all.

**Reality:** Production systems need reproducible retrieval.

**Why this matters:**

```
Scenario: User reports bug in RAG system
Engineer: "I can't reproduce - what version of the data were you using?"
User: "I don't know?"
Engineer: 🤯
```

Without versioning:

- Can't reproduce bugs
- Can't A/B test data changes
- Can't rollback bad updates

### How strong engineers version data:

python

```
class VersionedVectorDB:
 def __init__(self):
 self.current_version = self._get_latest_version()

 def _get_latest_version(self):
 versions = list(Path('data/versions').glob('v*'))
 if not versions:
 return 'v0'
 return max(versions).name

 def create_snapshot(self, version_name=None):
 """Create immutable snapshot of current data"""
 if version_name is None:
 version_name = f"v{int(self.current_version[1:]) + 1}"

 snapshot_dir = Path(f'data/versions/{version_name}')
 snapshot_dir.mkdir(parents=True)

 # Copy vector DB files
 shutil.copytree('data/vector_db', snapshot_dir / 'vector_db')

 # Copy document index
 shutil.copy('data/document_index.json', snapshot_dir / 'index.json')

 # Save metadata
 metadata = {
 'created_at': time.time(),
 'num_documents': self.db.collection.count(),
 'embedding_model': 'all-MiniLM-L6-v2',
 'chunking_strategy': 'semantic'
 }

 with open(snapshot_dir / 'metadata.json', 'w') as f:
 json.dump(metadata, f)

 return version_name
```

```

def load_version(self, version):
 """Load specific version for reproducibility"""
 version_dir = Path(f'data/versions/{version}')

 # Load vector DB from snapshot
 self.db = VectorDatabase(
 persist_directory=str(version_dir / 'vector_db')
)

 return version

def compare_versions(self, v1, v2, test_queries):
 """A/B test two data versions"""
 results_v1 = []
 results_v2 = []

 self.load_version(v1)
 for q in test_queries:
 results_v1.append(self.search(q))

 self.load_version(v2)
 for q in test_queries:
 results_v2.append(self.search(q))

 # Compare metrics
 return {
 'v1_avg_score': np.mean([r[0]['score'] for r in results_v1]),
 'v2_avg_score': np.mean([r[0]['score'] for r in results_v2])
 }

Usage
versioned_db = VersionedVectorDB()

Ingest new data
versioned_db.ingest_documents(new_docs)

Create snapshot before deploying
v3 = versioned_db.create_snapshot('v3')
print(f"Created snapshot {v3}")

A/B test
results = versioned_db.compare_versions('v2', 'v3', test_queries)
if results['v3_avg_score'] > results['v2_avg_score']:
 print("v3 is better, deploying")
else:

```

```
print("v3 is worse, rolling back to v2")
versioned_db.load_version('v2')
```

---

## How This Perspective Prevents Failures

### Fragile System (Amateur):

- Default LangChain settings
- One chunking strategy for all documents
- No embedding caching (re-embed on every run)
- No data versioning (can't reproduce bugs)
- Post-filter metadata (slow queries)
- **Result:** Works on 100 docs, breaks at 10,000

### Robust System (Engineer):

- Domain-specific chunking strategies
- Embedding pipeline with caching and deduplication
- Incremental updates (10x faster reindexing)
- Data versioning (reproducible, A/B testable)
- Pre-filter metadata with separate indices
- **Result:** Scales to millions of documents, debuggable, maintainable